

Nameless parameters and unutterable specializations

Document number: P0736R1
Date: 2018-05-05
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup: Evolution
Revises: P0736R0
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Changelog

Changes from R0

- Replaced conjunction with disjunction in the `int::value` example demonstrating unutterable specializations under Concepts.
- Replaced the `[name]` syntax, which need further refinement to resolve parsing issues, with a `for <...>` syntax.
- Added a note on the status of constraints in partial ordering.
- Added alternate placement of the *requires-clause* as a possible strategy.
- Added an example of explicitly specializing a member of a partial specialization.
- Added acknowledgements.

Issue/Background

When an expression is part of a signature for a function template, whether expressions are equivalent for the purposes of resolving declarations of the same entity to each other is defined by [temp.over.link]. This definition in turn relies on the form of the expression in the style of the one-definition rule ([basic.def.odr]).

Unfortunately, it is not specified what form an expression takes when template parameters are substituted with their corresponding arguments. Indeed, with the addition of *requires-clauses*, substitution might not occur in certain subexpressions. This frustrates the ability to form redeclarations in contexts where certain template parameters have no name, such as when an enclosing template is explicitly specialized. Redeclarations in such contexts also obfuscate the relationship between declarations expressed in terms of the primary template and declarations expressed with reference to specializations.

Current implementation behaviour

Even without the addition of Concepts, it is possible to encounter the problem of lacking a suitable way to form equivalent corresponding expressions in redeclarations.

For example, given:

```
template <unsigned N>
struct TA {
    template <unsigned M>
```

```
void f(unsigned (*)[N + M]);  
};
```

An attempt to produce an explicit specialization of `f` for `TA<0>` might yield:

```
template <>  
template <unsigned M>  
void TA<0>::f(unsigned (*)[0u + M]);
```

Indeed, it happens to work with both GCC and Clang. Unfortunately, not all literal types can necessarily have their values expressed as integer literals of the type (`short` comes to mind). In any case, it only happens to work and may stop working at any time. GCC (at least as late as version 7.2 from August 2017) would have accepted `0` in place of `0u` despite the type mismatch (so the `short` conundrum was somewhat sidestepped), but no longer does so.

Now with the addition of Concepts, an additional complication appears.

Given:

```
template <typename T>  
struct TB {  
    template <typename U>  
    void f() requires U::value || T::value;  
};
```

A similar approach to forming the explicit specialization of `f` for `TB<int>` would create the unutterable `int::value`. No implementation of Concepts known to the author accepts the utterance of `int::value`.

Possible solutions

It is the hope of the author that either a solution is adopted to ease the formation of related declarations or the effective inability to produce truly equivalent expressions in said contexts is made clear through examples to be incorporated into the Standard.

In the pursuit of a solution, syntax to allow binding of a name for specialized template arguments of the primary template could be explored.

In the context of the above cases, such syntax might look like the following (with the name appearing as a label in a context from which the argument of the specialization can be deduced):

```
template <>  
template <unsigned M>  
void TA<N: 0>::f(unsigned (*)[N + M]);  
  
template <>  
template <typename U>  
void TB<T: int>::f() requires U::value || T::value;
```

Notice that in the latter case, `T` would need to be treated as dependent.

Such a binding could also allow for partial specializations to have constraints expressed in terms of the parameters to the primary template. Below, in order to introduce the names before their use, the argument list for the specialization is introduced within the *template-head*; the *simple-template-id* then names the specialization using the bound names:

```

template <typename T, typename U>
struct A;

template <typename TT, typename UU> for <T: TT *, U: UU>
requires C<T, U>
struct A<T, U> { };

template <typename TT, typename UU> for <T: TT, U: UU *>
requires C<T, U> && C1<T>
struct A<T, U> { };

```

Without the binding, the intended subsumption of `C<T, U>` in the first specialization by the *requires-clause* of the second would be muddled; however, because partial ordering only considers constraints after deduction in both directions succeed, the subsumption does not come into play^[1].

In any case, the syntax for introducing names could also be used for explicit specializations of members of partial specializations; the binding list then binds arguments for the parameters of the partial specialization:

```

template <unsigned N>
struct A<TA<N + 1>, TA<N> > {
    template <unsigned M, int (*)[N]>
    TA<N + M> *f(unsigned (*)[N + M]);
};

template <> for <N: 42>
template <unsigned M, int (*)[N]>
TA<N + M> *A<TA<N + 1>, TA<N> >::f(unsigned (*)[N + M]) { return 0; }

```

Consideration needs to be given to that fact that the type associated with the introduced name is not known until it is used in the corresponding component of the *nested-name-specifier*.

Thus we speak the unutterable and name the nameless.

Further possibilities

Generalizing the `for <...>` syntax

The `for <...>` syntax can be generalized under a model where specializations are declared like redeclarations of the primary template, with the new syntax providing both a proxy of the template parameter list (with or without names of the parameters) for the “redeclaration” and the arguments of the specialization.

```

template <unsigned N> for <TA<N + 1>, TA<N> > struct A;
template <unsigned N> struct A<TA<N + 1>, TA<N> >; // redeclaration

```

Alternate placement of the *requires-clause*

The short syntax can be used for partial specialization with constraints by deferring the *requires-clause* until after the *simple-template-id*.

```

template <typename TT, typename UU>
struct A<T: TT *, U: UU> requires C<T, U>;

template <typename TT, typename UU>
struct A<T: TT, U: UU *> requires C<T, U> && C1<T>;

```

```
template <typename> int v;  
template <typename U> int v<T: U *> requires C1<T> = 0;
```

Acknowledgements

The author thanks Richard Smith for his input at the Jacksonville meeting of 2018 and his further contribution (including that of the `for <...>` syntax) in the preparation of this paper. As usual, any remaining mistakes are the responsibility of the author.

Notes

1. [^](#) This is necessary for partial ordering of function templates, since there is no established relationship in that case for constraints expressed in terms of the template parameters of one template to be matched with those expressed in terms of the template parameters of another template. The author believes that partial ordering of class template partial specializations can consider constraints without deduction succeeding in both directions when a mapping like the one proposed by this paper is present.